Analysis and Design of Algorithms

CS3230

Recitation

Week 6

Prof Lee Wee Sun

Sorting Algorithms

In-Place

A sorting algorithm is in-place if it uses very little additional memory beyond that used for the data, usually $O(1)$ or $O(\lg n)$.

Useful when memory is limited.

Examples:

- Insertion sort
- Quicksort ($O(\lg n)$ additional memory with certain implementations)

Stable

A sorting algorithm is stable if the original order of equal elements is preserved in the sorted output.

Used to sort each digit of radix sort.

Examples:

- Insertion sort
- Merge sort
- Counting sort

Linear Time Sorting

Counting Sort

- Runtime depends on input range
- $O(n + k)$ where k is the range of the input
- Linear time if k is $O(n)$
- Used to sort each digit of radix sort.

Linear Time Sorting

Radix Sort

- Runtime depends on input range.
- Run counting sort on each digit iteratively
- Need stable sort for sorting digits (usually counting sort)
- Digits usually lg n bits (check input size first, then use it to specify digit size)
  - Range of lg n bit is $O(n)$ so counting sort run in $O(n)$ time
- If range M, then range requires lg M bits and run lg M/lg n iterations.

Comparison Sorts

Use only comparison for all branching decisions.

Runtime does not depend on input range (assuming comparison takes constant time).

Examples:

- Insertion sort
- Merge sort
- Quicksort

Lower Bound for Comparison Sorts

- There are $n!$ possible permutations as outputs of binary decision tree
- A binary decision tree with $n!$ possible leaves has $\lg n! = \Omega(n \lg n)$ height
  - $\Omega(n \lg n)$ comparisons in the worst case
- Merge sort has runtime $\Theta(n \lg n)$ hence is an asymptotically optimal comparison sort
- Lower bound technique applies more generally for other problems using the decision tree model of computation.

Randomized Quicksort

- Quicksort has expected runtime of $\Theta(n \lg n)$ when all permutations of inputs are equally likely.
- But worst-case can be $\Omega(n^2)$, sorted inputs.
- Randomizing the pivot selection is equivalent to randomly permuting the inputs.
  - Expected runtime $\Theta(n \lg n)$
  - Even adversary cannot force worst case behaviour, as adversary cannot control randomization used.

Admin

- Reminder to complete Programming Assignment 1 by Friday midnight.
  - Submit directly to CodeCrunch (no need to submit on Coursemology)
- Midterm will be held on Saturday 1 October at 10am-11.30am in I3 Auditorium.
  - CLOSED book but 1 handwritten (both sides) crib sheet allowed.
  - Last year's midterm available on Coursemology under Materials -> Revision

MCQ

Question 1

There exists a comparison-based sorting algorithm that can sort any 6-element array using at most 9 comparisons.

- True
- False

Solution

There are $6! = 6 * 5 * 4 * 3 * 2 = 720$ permutations of 6 elements.

A complete binary tree of height 9 has only 512 leaves which is not enough to describe an algorithm that can output 720 permutations.

Any comparison sort algorithm requires at least 10 comparisons for some input.

Question 2

Suppose that an array contains n numbers, each of which is -1, 0 or 1. Then, the array can be sorted in $O(n)$ time in the worst case.

- True
- False

Solution

True. Can transform the numbers to positive integers by adding 2 to every number, do counting sort, then transform back by subtracting 2 to every number.

Or more simply, count how many -1, 0, and 1 there are in the array, then list out in order according to the counts. Why is the counting sort given in lecture somewhat complicated?

Question 3

Which of the following sorting algorithms (under typical implementations) will **not** work correctly as the auxiliary sorting algorithm (for sorting a digit) in radix sort?

- Insertion sort
- Merge sort
- Quicksort
- Counting sort
- Bubble sort

Solution

For use as an auxiliary sort to sort the digits in radix sort, the sorting algorithm needs to be stable.

Quicksort, under typical implementations, is not stable.

Question 4

Suppose we are given n strings which has at most 10 characters each. Which of the following method is asymptotically the fastest?

- Radix Sort
- Merge Sort
- Quicksort

Solution

Merge sort takes $\Theta(n \lg n)$ in the worst case.

Randomized quicksort takes $\Theta(n \lg n)$ expected time.

Since the range is a constant, radix sort takes $O(n)$ time.

Question 5

Let A_1 and A_2 be two sorted arrays of n real numbers (all distinct). In the comparison model, constructing a binary search tree of the set $A_1 \cup A_2$ requires $\Omega(n \log n)$ time.

- True
- False

Solution

Merge the two sorted arrays. $O(n)$ time

Build a binary search tree from a sorted array. $O(n)$ time.

Total time is $O(n)$.

Question 6

It is possible to sort an array A of n integers sampled randomly from a distribution in the range $[0..n^2-1]$ in linear time.

- True
- False

Solution

True. Use radix sort. Numbers within the range n^2-1 can be represented using 2lg n bits. Using digits of size lg n, we need to do 2 iterations of counting sort, where each iteration takes linear time.

Question 7

Who is the Master of Algorithms pictured below?

- Donald Knuth
- Robert Tarjan
- Andrew Yao
- Tony Hoare

Andrew Yao

- Turing Award Winner
- Professor and Dean at Tsinghua University
- Well known for work in theoretical computer science, cryptography, pseudo-random number generation, and also Yao's minimax principle that is used for proving lower bounds on performance of randomized algorithms

Structured Questions

Question 1(a)

Give one scenario when it is preferable to use merge sort instead of quicksort. Explain why.

Solution: When fast worst-case guarantee is required, e.g. for real-time applications, as merge sort has worst case runtime of $\Theta(n \lg n)$ while the worst-case guarantee for quicksort is $\Theta(n^2)$.

Question 1(b)

Give one scenario when it is preferable to use a comparison sort instead of radix sort. Explain why.

Solution: When the range of the numbers is very large (e.g. when sorting real numbers) as radix sort has a runtime that depends on the input range while a comparison sort runs in time O(n lg n) regardless of input range.

Question 1(c)

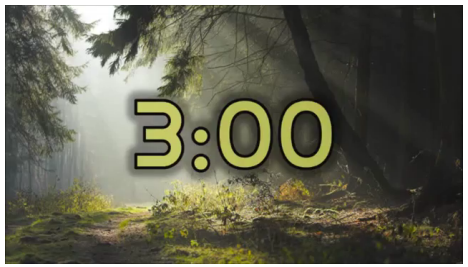
True, False, Justify: You need to sort n numbers. You find out that the range of those numbers is in $[1,2^n]$. You should use radix sort in preference to merge sort.

False. The runtime of radix sort is $\Theta\left(\frac{\lg M}{\lg n} n\right)$ where M is the range.

In this case, the runtime is $\Theta\ (n^2/\lg n)$ which is poorer than merge sort's runtime of $\Theta(n \lg n)$.

Question 2(a)

True, False, Justify: Under a decision tree model, any algorithm for merging two sorted lists of length n each takes at least n comparisons in the worst case.



Solution: True.

We argue that there are at least 2^n ways to construct the sorted list from the two sorted lists.

In the decision tree model, the height of the tree is at least log of the number of solutions, hence at least n.

To count the number of ways to merge the lists, note that there are $2n$ choose n ways to place the first array within the merged array.

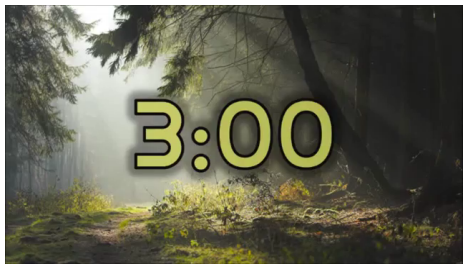
$$\binom{2n}{n} = \frac{2n(2n-1)\ldots(n+1)}{n(n-1)\ldots 1}$$

$$= \left(\frac{2n}{n}\right)\left(\frac{2n-1}{n-1}\right)\ldots\left(\frac{n+1}{1}\right)$$

$$\geq 2^n$$

Question 2(b)

Ali has 81 coconuts, all of which have the same weight, except for one which is heavier. He does not know which is the heavier coconut. Ali's friend has a balance scale, but will charge Ali one dollar for each use of the scale. What is the maximum amount of money that Ali has to pay to guarantee that he can find the heaviest coconut, assuming that Ali uses the optimal algorithm?



Solution: Divide the coconuts into three equal sized piles and put two of the piles on the two sides of the scale. If they are equally heavy, repeat with the third pile, otherwise repeat with the heavier pile, until the heavy coconut is found.

For 81, coconuts, we get the sequence of 27, 9, 3, and 1 coconuts, allowing us to identify the heavy coconut in 4 weightings.

To see that this is optimal, note that the scale can divide the coconuts into at most 3 groups with each weighting. Any algorithm using only the scale can be described as a ternary tree with the label of the heavy coconut at each leaf. A full ternary tree of height 3 has only 27 leaves, hence there is no algorithm with the worst case number of weightings of 3 or fewer.